

Practice #3: Ensuring that retrieving frequently used data like the user's session state is faster, and does not require too many round trips to resources like the database by using caching techniques.

In load balancing, whenever the server side sessions are not practical, the application needs to store the session state in some other data store, from where the sessions can be fetched quickly.

The options to store such session states could be:

1. Store it in the client side, in the form of cookies, or HTML5 storage APIs.
2. Store it in a database. Most databases offer to store session states, so use the feature.

Caches are based on the 'Locality of reference' principle; recently requested data is likely to be requested again. Every layer of the Web application architecture can have the caching of data or caching of static resources.

The most often used caches are at the front-end tier. The idea is to use them in close proximity to the front end or application logic to make sure the response to a request has the least hops possible.

The most popular caches are Memcache and Redis cache. These can be hosted on separate clusters and calls can be made to them using a particular protocol at a particular port. The cache cluster is expandable in its own sense. Most of the caches work on a key-value store basis and keep data in memory (RAM, usually), since the data is retrieved against a key.

Both Memcache and Redis can serve local as well as distributed caches. Many large websites like Facebook use Memcache clusters for their caching mechanisms (Reference: '*Scaling Memcached at Facebook*').

For cloud implementations, one can choose between various options — either Redis, Memcache or proprietary protocol based caches.

Table 1 gives a comparative analysis of the above discussed caching technologies.

Table 1

Technology	Pros	Cons
Memcache	Key-Value O(1) Horizontal scaling is much easier	Data persistence is not possible Max value: 1MB Max Key: 250 bytes
Redis	Offers replication Configurable data persistence Key/value: Up to 512MB	
Cloud service caches	Choose from Redis/Memcache; Easily scalable	

Practice #4: Store large amounts of content in a separate

data repository, which can be scaled separately.

Scaling the storage thus is our concern at this point. The golden rule of scaling is 'Decentralise to scale'.

So, at this point, let's focus on decentralising the storage and taking it off from the monolithic application server approach. To do that, let's start using a scalable storage which, if running in an on-premises data centre, could be a NAS (network attached storage). In this case, one can add more and more storage to a NAS to scale it independently. There are other open source implementations, too, that can be used for on-premise virtual appliances, like OpenStack Swift or CEPH.

Figure 5 shows what our architecture looks like after the above mentioned change.

Practice #5: Ensure low latency while videos/documents are being viewed and used, by using geo-redundant copies of content in different geographies.

Having NAS will not ensure the reduction of latency while downloading the content. Even having cloud based storage will not ensure the same at the first go, unless it is configured to do so.

So the answer to low latency, which will also help in achieving high availability, is to go for geographically redundant storage copies, so that the requests coming from different parts of the world get served from the geographically nearest data centre. You can also consider using a CDN (content delivery network), which ensures low latency by keeping redundant copies.

Table 2 gives a comparative analysis of the above discussed storage options.

Table 2

Technology	Pros	Cons
Network attached storage	Easily scalable Resilient Data redundancy Data encryption User access control	Need to buy more storage disks every time a Web app scales up Maintenance overheads
Cloud service storage/CDN	Easily scalable Pay-as-you-go Samba protocol (optional) Geographic redundancy Options to move to cold data stores	Pay extra for redundancy

However, you need to keep the following points in mind:

1. While working with cloud based storage, you pay for the data stored even if it is not used. This is called Cold Data. If the use case allows moving such data to a Cold Data store, then it can help save storage costs to an extent.